Nikhil Savant

312-989-8206 | nsavant033@gmail.com | linkedin.com/in/nikhil-savant | U.S. Citizen

Education

University of Illinois Urbana-Champaign — Urbana-Champaign, IL
Bachelor of Science in Computer Science and Economics — *Expected Graduation: 2027*
Coursework: Database Systems, Algorithms and Models of Computation, Computer Systems, Data Structures

Technical Skills and Clubs

Languages: Java, Python, Go, TypeScript, C++, C, JavaScript, R, HTML/CSS, Swift, SQL
Frameworks: React, Node.js, Lit, Spring Boot, NextJS, React Native, LangChain
Developer Tools: Linux, Docker, Kubernetes, AWS, Jupyter, Git, MongoDB, Figma, Ansible, Nginx, Ollama, ChromaDB
Libraries: PyTorch, Pandas, PEFT, OpenCV, NumPy, SK-Learn
Clubs: UIUC ACM Math and Algorithms Member, Engineering Council Activities Coordinator

Work

Cisco — May - Aug 2026
Software Engineer Intern — San Jose, CA

- Built an AI agent over 9 live infrastructure tools via a TypeScript MCP gateway wrapping internal REST APIs as JSON-Schema-validated tools, standardizing discovery, argument validation, typed responses, and error handling
- Implemented delegated authorization using signed JWT claims and controller-level RBAC, validating and propagating caller identity/scopes before each MCP tool execution and preventing privilege escalation
- Engineered a repeatable RAG ingestion pipeline, embedding **2,300+** documents into **13,000+** vector-indexed chunks for semantic retrieval with deterministic source citation
- Built a 100-case agent evaluation suite measuring tool-selection accuracy, argument validity, retrieval Recall@K, citation correctness, and task success rate, improving end-to-end pass rate from **74% to 88%**

Algo Analytics — Jun - Oct 2025
Full-Stack Engineer Intern — Remote

- Developed a TypeScript-based backend integration layer to connect the React Native client with financial market APIs, aggregating real-time stock data and reducing client-side parsing times by **30%**
- Implemented REST API controllers to validate and serialize market data, processing **100,000+** data points daily while achieving a **300ms** API response time and trimming JSON payloads by **20%** by pruning unused response fields
- Added JWT Authentication with silent token refresh through client-side Axios interceptors, eliminating re-logins during active sessions

StellarPay — Jun - Aug 2025
Software Engineer Intern — Remote

- Built a GPT-powered financial assistant in TypeScript/Node.js using RAG, ingesting documents with AWS Textract and indexing OpenAI embeddings in Pinecone to raise retrieval precision by **40%**
- Refactored front-end components into shared modules, leading to a reduction of code duplication by **25%** and in page load times by **20%**
- Secured sensitive user data with IAM role-based access control, KMS-encrypted storage (AES-256), and JWT authorization on API Gateway

Projects

Spotify Music Recommender

- Built a personalized music recommendation system that learns from 52M public listening events and creates a new Spotify playlist with the recommendations
- Developed an ETL pipeline that aggregates raw listening data into a user-item interaction matrix and resolves inconsistent track metadata across data sources
- Applied collaborative filtering through matrix factorization (ALS) with hyperparameter optimization and ranking-based evaluation, ensembling it with an item-similarity model to raise NDCG@10 from **0.30 to 0.71**
- Modeled shifting user taste by detecting recent listening trends and weighting them against long-term preferences, raising NDCG@100 from **0.05 to 0.11** on the user's newest additions

Personal AI Assistant

- Built a preprocessing pipeline converting 450+ of my iMessage conversations into 5,000 context–reply training examples, splitting by conversation session to prevent overlapping context across training and validation
- Fine-tuned Qwen2.5-1.5B-Instruct using QLoRA, applying reply-only loss and reducing held-out reply perplexity by **25%** compared to the base model
- Deployed the assistant on a self-hosted Linux server by merging the adapters and quantizing to 4-bit GGUF, serving it through Ollama and Open WebUI at **25 tokens/sec** with under 0.9s to the first token